Classification Pipeline

Large-Scale Text Classification under Strict Resource and Parameter Constraints

Dataset:	`dataset_10M.parquet` — 10,000,000 rows (∼4 GB)
Task:	24-class text topic classification
Constraints:	No pretrained models; $\leq$ 5B parameters; local hardware only
Final Model Size:	3,859,608 parameters
Final Macro F1:	0.5244

a. Data Processing

Data Loading Strategy

The provided dataset (`dataset_10M.parquet`) consists of 10,000,000 rows and totals approximately 4 GB. Loading this entirely into memory using standard libraries like Pandas results in catastrophic Out-Of-Memory (OOM) failures. To engineer around this constraint, the following strategy was adopted:

- **Exploratory Data Analysis (EDA):** `Polars` with lazy evaluation was used to scan the dataset and compute class distributions and summary statistics out-of-core.
- **Streaming Training Pipeline:** For model training, a custom PyTorch `IterableDataset` powered by `pyarrow.parquet` was built. This pipeline streams the dataset from disk in mini-batches (row groups), maintaining a near-zero memory footprint and allowing the model to train on the full 10 million rows continuously on standard hardware.

Preprocessing & Tokenization

Adhering strictly to the constraint forbidding pretrained models, a custom Byte-Pair Encoding (BPE) tokenizer was engineered from scratch.

- **Feature Engineering:** Raw text (`DATA`) was pre-tokenized using a standard whitespace splitter, followed by BPE merging to capture subword representations efficiently.
- **Vocabulary & Padding:** The tokenizer was trained on a randomized 500,000-row sample to prevent overfitting to early rows. The vocabulary size was explicitly capped at 30,000 to restrict the size of the model's embedding layer. Sequences were truncated or padded to a fixed `max_len` of 256 tokens.

c. Exploration & Iteration

The Experimentation Journey

The primary challenge was achieving high classification metrics without violating the 5-billion-parameter limit or relying on massive Transformer architectures that would bottleneck local hardware.

Experiment 1 — Baseline Global Average Pooling

Approach

A lightweight Global Average Pooling (GAP) network was trained with a standard Cross-Entropy Loss function.

Observation

The model successfully trained but exhibited a severe performance skew, achieving an impressive 86.99% Accuracy alongside a low Macro Recall of 45.27%.

Analysis of Failure

EDA revealed extreme class imbalance: the top three topics comprised over 44% of the dataset, while the bottom tier contained less than 1%. The standard loss function rewarded the model for aggressively predicting majority classes and ignoring rare topics (e.g., `games`, `industrial`), artificially inflating overall accuracy while destroying the F1 score.

Experiment 2 — Dynamic Class Weighting (Final Selection)

Approach

Rather than naively increasing the model's parameter count (which risks overfitting and slow inference), a mathematical intervention was engineered. Using Polars, the exact inverse frequencies for all 24 classes were computed across the 10 million rows and injected as a dynamic weight tensor into the `CrossEntropyLoss` criterion to heavily penalize majority-class bias.

Formula utilized:

$$w_i = \frac{N}{C \times n_i}$$

where N is the total number of samples, C is the number of classes, and n_i is the count of class i.

Result

This intervention worked as intended. Macro Recall increased to 61.85%, and Macro F1 improved to 52.44%. The modest expected drop in overall accuracy (to 83.80%) was a mathematically sound trade-off to achieve a robust classifier capable of identifying heavily underrepresented topics.

Table 1: Comparison of Experiment 1 vs. Experiment 2

Metric	Exp. 1 (Baseline)	Exp. 2 (Weighted)
Accuracy	**86.99%**	83.80%
Macro Recall	45.27%	**61.85%**
Macro F1	—	**52.44%**

e. Architecture Details (Final Model)

Model Structure

The `CustomTextClassifier` is an efficient, feed-forward pooling network constructed using base `torch.nn` modules:

1. **Embedding Layer:** (30,000 vocab × 128 dim) — maps subword tokens to dense vectors.
2. **AdaptiveAvgPool1d:** Acts as a Global Average Pooling (GAP) layer, flattening the sequence dimension to capture the global semantic meaning of the document.
3. **Classification Head:** Linear (128 → 128) → ReLU → Dropout (0.3) → Linear Output (128 → 24 classes).

Parameter Count

The model contains exactly **3,859,608 parameters**. This explicitly and safely satisfies the constraint to remain under the 5B limit while retaining enough structural complexity to capture textual nuances.

Design Decisions and Trade-Offs

LSTM and Transformer blocks were deliberately avoided. While self-attention captures sequential word order, it scales quadratically with sequence length ($\mathcal{O}(N^2)$), making a 10-million-row

training run computationally unfeasible on local hardware within a strict time constraint. The GAP architecture ($\mathcal{O}(N)$) sacrifices strict word order but acts as a highly optimized continuous bag-of-words (CBOW) feature extractor, providing rapid convergence and ultra-fast inference.

f. Training Strategy

- **Train/Validation Split:** The streaming nature of the dataset was leveraged by training continuously on the sequential Parquet file. Evaluation was conducted on a distinct 10,000-row streaming validation sample.
- **Hardware Used:** Local CPU.
- **Efficiency Considerations:** By circumventing DataLoader memory spikes using row-group streaming, the pipeline successfully processed the entire 10,000,000-row dataset in a single uninterrupted pass.

Table 2: Training Hyperparameters

Hyperparameter	Value
Optimizer	AdamW (weight decay for regularization)
Learning Rate	1e-3
Batch Size	128
Sequence Length	256
Random Seed	`set_seed(42)`

g. Evaluation Metrics

Tested on a deterministic 10,000-row validation sample, the final weighted model achieved the following results:

Table 3: Final Model Evaluation Metrics

Metric	Score
Accuracy	0.8380
Precision (Macro)	0.5295
Recall (Macro)	0.6185
F1 Score (Macro)	0.5244

h. Error Analysis

Where the Model Fails & Misclassification Patterns

The detailed per-class classification matrix reveals a direct mathematical consequence of the class-weighting intervention: a distinct precision/recall trade-off on minority classes.

Over-Sensitivity

For exceptionally rare classes like `industrial` (only 36 examples in the validation batch), the model achieved a staggering 0.92 Recall but a low 0.07 Precision. The dynamic weights forced the model to become highly sensitive to minority topics, causing it to over-predict them (generating false positives) to guarantee they were not missed.

Semantic Overlap

The model continues to struggle with classes that share heavily overlapping lexicons. For example, `art_and_design` and `entertainment` frequently share terminology, leading to lower isolated F1 scores (0.29 and 0.40, respectively). Without multi-head attention to determine exact contextual syntax, the GAP layer occasionally blends these semantic boundaries.

Potential Improvements

To maintain high minority-class recall while recovering precision, future iterations would replace the static inverse-frequency weights with **Focal Loss** ($\gamma = 2$). This would dynamically scale the loss based on the model's prediction confidence, preventing it from continuously over-predicting rare classes once it has confidently learned their base features. Additionally, scaling the embedding dimension from 128 to 256 could provide better geometric separation for overlapping semantic topics.